GAME CLIENT PORTFOLIO · SOLO PROJECT

gun-project

3D 슈팅 액션 · 2D 로그라이크 — 1인 개발 프로토타입 2종

Unity 2022.3 LTS | C# | NavMesh(AI Navigation) | Rigidbody2D | Animator | uGUI | Git Flow(PR)

김동준 · dkdl01028@icloud.com · github.com/kimdongjun22

기간	2025.05.29 ~ 2025.06.04 (약 1주)	**인원**	1인 개발
담당	기획 · 프로그래밍 · 레벨 구성 전체	**저장소**	github.com/kimdongjun22/gun-project

1인	**2종**	**19개**	**15회**
기획·개발 전 과정	3D 슈터 / 2D 로그라이크	직접 작성한 C# 스크립트	develop → main PR 머지

프로젝트 소개

한 저장소에서 장르가 다른 Unity 프로토타입 두 개를 혼자 기획하고 구현했습니다. 같은 "사격 + 적 추적" 게임 루프를 **3D(`gun`)와 2D(`roglike1`)로 각각 만들어 보면서**, 물리·입력·AI가 두 환경에서 어떻게 다르게 구현되는지 익히는 것이 목표였습니다.

프로젝트	장르	구현된 게임 루프
gun (3D)	도시 배경 슈팅 액션	이동 → 마우스 사격(히트스캔/투사체) → 적 처치 → 아이템 획득·점수, 쿨다운 범위 스킬, 메인 메뉴 → 게임 씬 전환
roglike1 (2D)	탑다운 로그라이크	8방향 이동·애니메이션 → 근접한 적이 추적 → 장착 무기로 투사체 발사, 시작 시 랜덤 던전(방) 생성

담당 구현 내용

gun — 3D 슈팅 액션

- **두 가지 사격 방식**: 카메라 기준 `Physics.Raycast` **히트스캔 사격**(`Gun`)과 프리팹을 생성하는 **투사체 사격**(`PlayerController.Fire`)을 모두 구현하고 비교했습니다.
- **두 가지 적 AI**: `NavMeshAgent.SetDestination`으로 지형을 고려해 추적하는 `EnemyFollow`, `Vector3.MoveTowards`로 직선 추적하는 `Monster`
- **쿨다운 범위 스킬**: `Time.time` 차이로 5초 쿨다운을 계산하고, 반경 안의 몬스터에게 피해를 줍니다.
- **GameManager 싱글톤 + UI**: 점수와 체력을 한곳에서 관리하고 `Text` / `Slider` UI를 갱신합니다. 아이템(`PickupItem`)은 트리거로 획득합니다.
- **카메라·씬 흐름**: `LateUpdate` + `Lerp` 오프셋 추적 카메라, `MainMenuManager`의 씬 전환
- **레벨 구성**: POLYGON City 등 외부 에셋을 import해 도시 맵, 캐릭터, 총기, 애니메이션을 구성했습니다.

roglike1 — 2D 로그라이크

- **상속 기반 무기 시스템**: 추상 클래스 `Weapon`(`firePoint`, `abstract Fire()`)을 `Gun`이 구현하고, `PlayerWeapon.EquipWeapon()`이 무기 프리팹을 교체합니다. **새 무기는 `Weapon`만 상속하면 추가**됩니다.
- **물리 기반 2D 이동**: 입력은 `Update`에서, 이동은 `FixedUpdate`의 `Rigidbody2D.MovePosition`에서 처리하도록 분리했습니다. 애니메이터 블렌드 파라미터(Horizontal/Vertical/Speed)도 연동했습니다.
- **RPG 스탯 구조**: HP, 이동속도, AD/AP, 회피, 치명타 스탯과 기본값 백업(`Player`)
- **거리 기반 적 AI**(`EnemyAi`), **랜덤 방 배치 던전 생성**(`DungeonGenerator`), ESC 종료 처리

대표 코드

① 추상 클래스로 확장 가능한 무기 구조 — roglike1 / Weapon.cs · Gun.cs

```
public abstract class Weapon : MonoBehaviour
{
    public Transform firePoint;
    public abstract void Fire();          // 무기마다 발사 방식만 다르게 구현
}

public class Gun : Weapon
{
    public GameObject projectilePrefab;
    public float fireForce = 10f;

    public override void Fire()
    {
        GameObject bullet = Instantiate(projectilePrefab, firePoint.position, firePoint.rotation);
        bullet.GetComponent<Rigidbody2D>().velocity = firePoint.right * fireForce;
    }
}
```

② Raycast 히트스캔 사격 — gun / Gun.cs

```
void Shoot()
{
    muzzleFlash.Play();
    RaycastHit hit;
    if (Physics.Raycast(fpsCam.transform.position, fpsCam.transform.forward, out hit, range))
    {
        Enemy enemy = hit.transform.GetComponent<Enemy>();
        if (enemy != null)
            enemy.TakeDamage(damage);
    }
}
```

③ 시간 기반 쿨다운 스킬 — gun / PlayerController.cs

```
void UseSkill()
{
    if (Time.time - lastSkillTime >= skillCooldown)   // 코루틴 없이 타임스탬프로 쿨다운 판정
    {
        Skill.Activate(transform.position);
        lastSkillTime = Time.time;
    }
}
```

개발 과정

날짜	작업
05.29	프로젝트 생성, 맵(도시)과 캐릭터 에셋 구성, 플레이어 이동과 추적 카메라
05.30	`GameManager`, 히트스캔 `Gun`, `Enemy`, NavMesh 추적 AI, 아이템 픽업
06.02	`Monster`, 쿨다운 범위 스킬, 메인 메뉴 씬, 투사체 사격
06.03	2D 로그라이크 프로젝트 생성, 2D 캐릭터와 타일맵 구성
06.04	2D 이동·적 AI, 상속 기반 무기 시스템, 랜덤 던전 생성

모든 기능은 `develop` 브랜치에서 작업한 뒤 **PR 15건으로 `main`에 머지**했고, `[FEAT]` 커밋 컨벤션을 지켰습니다.

회고 및 개선 계획

- **배운 점**: 같은 기능을 3D와 2D로 각각 구현하면서 `Physics` / `Physics2D`, `Rigidbody` / `Rigidbody2D`의 차이와 NavMesh 사용법을 직접 비교했습니다. 무기 시스템을 추상 클래스로 설계하면서 **확장을 고려한 구조**의 장점을 체감했습니다.
- **개선할 점 1 — 2D/3D API 혼용**: 3D 프로젝트의 범위 스킬이 `Physics2D`를 쓰고 있습니다. `Physics.OverlapSphere`로 통일하겠습니다.
- **개선할 점 2 — 스탯 구조**: 스탯을 `static`으로 선언해 여러 적이 값을 공유합니다. 인스턴스 필드 또는 `ScriptableObject` 데이터로 분리하겠습니다.
- **개선할 점 3 — 중복 클래스**: 역할이 겹치는 `Enemy` / `Monster`는 공통 `IDamageable` 인터페이스로 정리하겠습니다.
- **개선할 점 4 — 저장소 관리**: 하위 폴더 프로젝트에 `.gitignore`가 적용되지 않아 `Library` 폴더가 커밋됐습니다. 이 경험으로 **Unity 프로젝트 루트마다 `.gitignore`를 두어야 한다는 점**을 배웠습니다.